Шпаргалка Neovim: клавиши и команды

Конфигурация для Windows 11: Python, Rust, PowerShell, локальный ИИ-помощник · октябрь 2026

Как читать

- `Esc` включает обычный режим (команды), `i` включает режим вставки (печать). Если не сказано иное, клавиши ниже нажимаются в обычном режиме.
- «Пробел a» означает: нажать пробел, отпустить, нажать `a`.
- Всё, что начинается с двоеточия, это команда: набрать её и нажать Enter.
- Пути в PowerShell можно писать с прямыми слэшами: `$env:LOCALAPPDATA/nvim` работает так же, как с обратными.

Режимы

Клавиши	Что делает
`i`	Режим вставки перед курсором
`o` / `O`	Новая строка ниже / выше и режим вставки
`Esc`	Выход в обычный режим
`jk` (в режиме вставки, быстро)	То же, что `Esc`
`V`	Выделить строки целиком (затем `j` / `k` растягивают выделение)
`v`	Выделить по символам

Сохранение и выход

Команда	Что делает
`:w`	Сохранить
`:wq`	Сохранить и выйти

Команда	Что делает
`:q`	Выйти (не сработает, если есть несохранённые изменения)
`:q!`	Выйти без сохранения
`:qa`	Закрыть все окна
`:qa!`	Закрыть все окна без сохранения (выручает, когда открыты чат или дерево)
`:e!`	Выбросить несохранённые изменения и вернуть файл как при последнем сохранении

Перемещение

Клавиши	Что делает
`j` / `k`	Вниз / вверх
`0`	В начало строки
`gg`	В начало файла
`G`	В конец файла
`:30` или `30G`	На строку 30
`Ctrl+Home`	В начало файла
`Ctrl+w`, затем `w`	Переключиться между окнами (файл, дерево, чат)

Поиск и правка текста

Клавиши	Что делает
`/слово`, Enter	Найти слово в файле
`:44d`	Удалить строку 44
`:25,$d`	Удалить строки с 25 до конца файла
`:%s/старое/новое/`	Заменить текст в файле (первое совпадение в каждой строке)

Клавиши	Что делает
`cw`	Заменить слово под курсором: удаляет слово и включает вставку
`0`, затем `D`	Очистить строку целиком (`D` удаляет от курсора до конца строки)
`gg=G`	Выровнять отступы во всём файле
`u`	Отменить последнее действие (нажимать можно несколько раз)
`Ctrl+r`	Вернуть отменённое

Файлы и поиск

Клавиши	Что делает
`:e $MYVIMRC`	Открыть init.lua
`:e $LOCALAPPDATA/nvim/lua/config/lazy.lua`	Открыть файл по пути
`Ctrl+p`	Поиск файла по имени (вместо стандартной клавиши)
`Ctrl+f`	Поиск текста по всем файлам проекта (вместо прокрутки страницы)
`Ctrl+n`	Открыть и закрыть файловое дерево слева
Стрелки, Enter, Esc	В окнах поиска и в дереве: выбрать, открыть, закрыть

Файлы конфигурации лежат в `$LOCALAPPDATA/nvim/`: `init.lua`, `lua/config/options.lua`, `lua/config/lazy.lua` (плагины), `lua/config/lsp.lua` (языковые серверы), `lazy-lock.json` (версии плагинов).

Код: подсказки и автодополнение

Клавиши	Что делает
`gd`	Перейти к определению функции или переменной (Python, Rust, PowerShell)
Текст справа от строки	Ошибки и предупреждения от языкового сервера
`E:2 W:1` в строке статуса	Количество ошибок и предупреждений в файле
Меню при наборе	Автодополнение показывается само
`Ctrl+y`	Принять выбранный вариант
`Ctrl+n` / `Ctrl+p` или стрелки	Выбрать вариант в меню
`Ctrl+e`	Закрыть меню
`Ctrl+Space`	Вызвать меню вручную

Примечание: в стандартной раскладке плагина blink.cmp клавиша Enter вариант не принимает, для этого нужен `Ctrl+y`.

Сервер запускается через 10-30 секунд после открытия файла. Если подсказок нет, выполни `:checkhealth vim.lsp`.

ИИ-помощник (локальная модель)

Перед работой Ollama должна быть запущена (значок ламы в трее). Модель: `qwen2.5-coder:7b`, контекст 16384 токенов.

Клавиши	Что делает
Пробел a	Открыть и закрыть чат
`V`, `j` / `k`, затем пробел c	Отправить выделенный код в чат
`V`, `j` / `k`, затем пробел i	Правка выделенного кода по твоей задаче
`i`, затем `Ctrl+s` (в чате)	Начать печатать и отправить вопрос
`g2` / `g3` / `g1`	Принять / отклонить / всегда принимать правку
`ga` (в чате)	Выбрать другую модель

Команды: `:CodeCompanionChat` (открыть чат), `:CodeCompanionChat Toggle`, `:CodeCompanionChat Add` (добавить выделение), `:CodeCompanion текст задачи` (правка на месте).

Вопрос по коду. Выделить код (`V`, `j` / `k`), пробел c, `i`, вопрос, `Ctrl+s`. Ответ приходит через 10-20 секунд.

Правка кода. Выделить код, пробел i, написать задачу, Enter. Правка появится отдельным блоком: `g2` принять, `g3` отклонить. Результат всегда стоит просмотреть: 7B-модель иногда ошибается в деталях (например, ставит комментарий не туда).

Совет: выделяй нужную функцию, а не весь файл. Так ответы точнее, а контекст не переполняется. Проверка работы на видеокарте: `ollama ps` (должно быть `100% GPU`).

Диагностика и служебные команды

Команда	Что делает
`:checkhealth`	Общая проверка состояния
`:checkhealth vim.lsp`	Работают ли языковые серверы
`:checkhealth lazy`	Состояние менеджера плагинов
`:checkhealth nvim-treesitter`	Установленные парсеры подсветки
`:checkhealth vim.provider`	Проверка провайдеров (Python, Node и др.)
`:Inspect`	Чем подсвечено слово под курсором (подтверждает работу Treesitter)

Команда	Что делает
`:set ft?`	Какой тип файла определил Neovim
`:TSInstall python`	Установить парсер подсветки для языка (также `rust`, `lua`)
`:Lazy`	Окно менеджера плагинов; `q` закрывает его

Команды PowerShell, которые использовались

Команда	Зачем
`nvim файл`	Открыть файл в Neovim
`nvim $env:LOCALAPPDATA/nvim/lua/config/lsp.lua`	Открыть файл конфигурации
`pwsh --version`, `rustc --version`, `cargo --version`	Проверить установленные программы
`rust-analyzer --version`, `pyright --version`	Проверить языковые серверы
`cargo new rust_test`	Создать тестовый проект на Rust
`nvidia-smi`	Объём видеопамяти и загрузка видеокарты
`ollama pull qwen2.5-coder:7b`	Скачать модель
`ollama run qwen2.5-coder:7b`	Задать вопрос модели в терминале
`ollama ps`	Какие модели загружены и на чём работают
`git init`, `git add .`, `git commit -m 'текст'`	Создать репозиторий и сохранить версию конфига
`git log --oneline`, `git status`	История и состояние репозитория

Частые проблемы

- **E37: No write since last change.** Файл не сохранён. Нажми `:wq` или `:q!`.
- **E486: Pattern not found.** Искомого текста нет в текущем файле или окне. Часто поиск запускают не в том файле.

- **Чат или команды «ломаются».** Мешает Punto Switcher. Настройки → Программы-исключения → добавить `WindowsTerminal.exe`.
- **Чат не даёт печатать.** Он открывается в обычном режиме. Нажми `i`.
- **`:q` не закрывает окно при открытых чате или дереве.** Используй `:qa!`.
- **Parser could not be created for язык.** Не установлен парсер. Выполни `:TSInstall язык`.
- **Подсказок кода нет.** Подожди 10-30 секунд и выполни `:checkhealth vim.lsp`.
- **Ответ модели не приходит.** Проверь, что Ollama запущена (значок в трее, `ollama ps`).